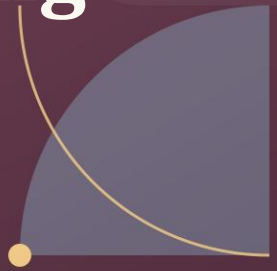
BRAIn lab
basic research of artificial intelligence

LECTURE 7

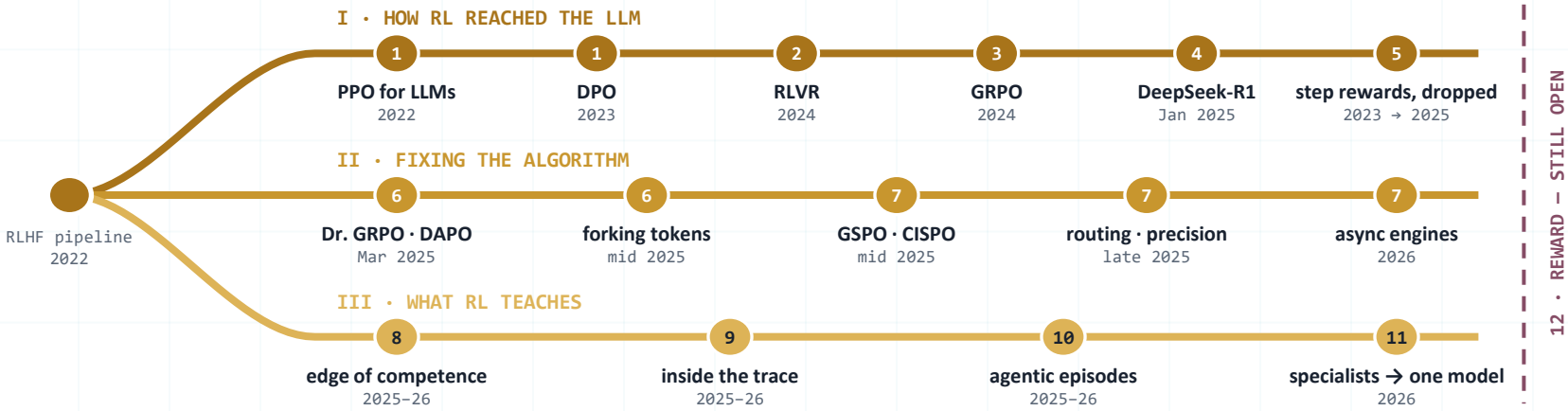
# Advanced RL & Reasoning



Sample a group of answers, compare them, shift the probabilities

**Denis Shveykin · 19 October 2026**

# Map of the Lecture




**MAP**

Each step fixes what the previous one exposed. The branches overlap in time; within a branch we go by date.

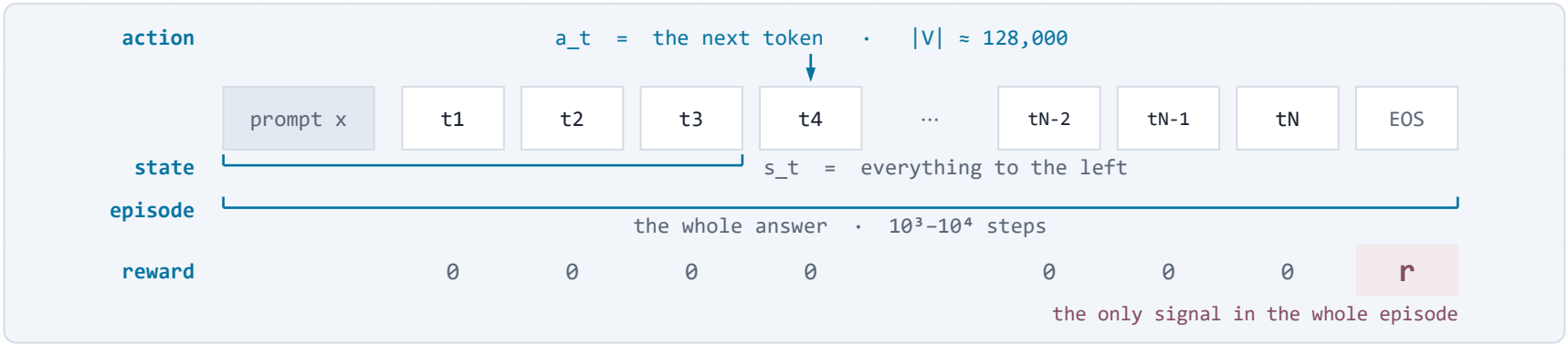
# 01

# How RL Reaches the LLM

Act I · the vocabulary and its two consequences



# The RL Vocabulary in Language-Model Terms




## Horizon — thousands of steps

## Reward — one scalar, at the very end

**FIG 1.1** Everything in this lecture follows from these two facts.

# PPO: Four Models in Memory

$$\mathcal{J}(\theta) = \mathbb{E}\left[\min\left(\rho_t A_t,\ \text{clip}(\rho_t, 1-\varepsilon, 1+\varepsilon)\, A_t\right)\right]$$

$$\rho_t = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}$$ computed per token

A_t comes from here

**actor**
$\pi_\theta$
the model we are training
TRAINED **112 GB**

**reference**
$\pi_{\text{ref}}$
frozen anchor for the KL term
FROZEN 14 GB

**reward model**
$r_\phi$
turns an answer into one scalar
FROZEN 14 GB

**critic**
$V_\psi$
the value of every prefix
TRAINED **112 GB**

all numbers for a 7B model · bf16 + Adam · same-size reward model · no sharding, no offload
frozen = 2 bytes/param · trained = 16 (weights + grads + fp32 master + Adam moments)

**252 GB**
**for a 7B policy — just to hold the four models**
activations and rollout KV cache on top

FIG 1.2 Two of the four are trained. Block 3 deletes one of them.

# To Judge a Prefix You Have to Finish It

How many distinct arrangements of the letters in BANANA?

six letters: three A, two N, one B
— so the count is 6! divided by

identical, byte for byte

the critic must return one number here

$$3! \cdot 2! \cdot 1! = 720/12 = 60$$

R = 1

the branches differ in one character

$$3! + 2! + 1! = 720/9 = 80$$

R = 0

$$V(s_t) = \mathbb{E}_{y \sim \pi(\cdot|s_t)}\left[\mathbf{1}[\text{correct}]\right]$$

FIG 1.3 Fluency, format and confident tone are all readable here. None of them tell the two branches apart.


Critic, what is this
half-written proof worth?
Difficult to see. Always
in motion is the future.

# DPO: No Reward Model, No Rollouts

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}\left[\log \sigma\left(\underbrace{\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)}}_{\hat{r}(x,y_w)} - \underbrace{\beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}}_{\hat{r}(x,y_l)}\right)\right]$$

the policy is the reward model — and σ sees only the gap

## What it removes

The reward model and the RL loop. Two models left — the policy and a frozen reference — and no generation at all.

126 GB for a 7B policy, against 252 before

## Likelihood displacement

Prefer No over Never: P(No) collapses 0.85 → 0.01 and Yes rises instead. One prompt, single-token answers, OLMo-1B.

Razin et al., ICLR 2025 · arXiv 2410.08847

## Pairs never move

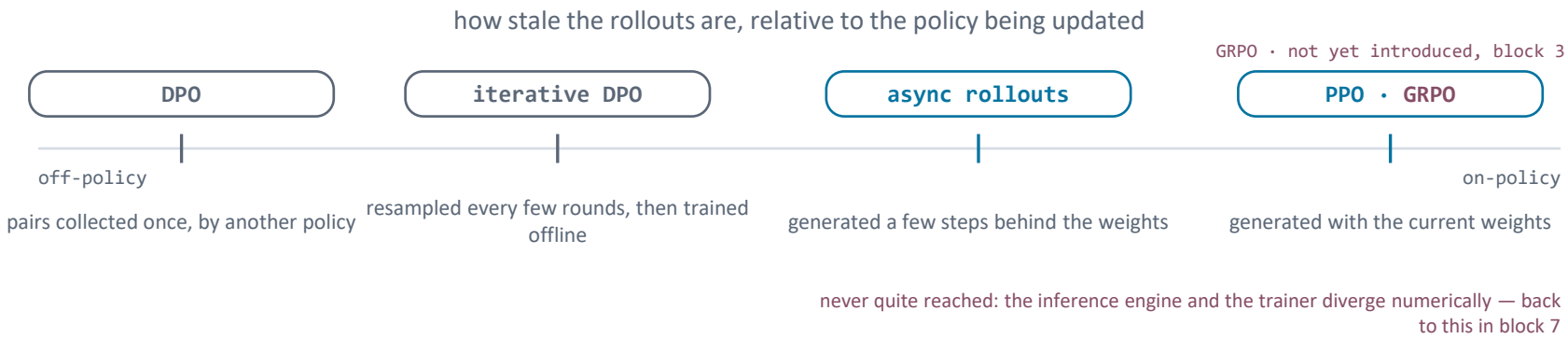
The comparisons came from another policy and stay fixed while θ moves. Nothing resamples them.

→ the on/off-policy scale, next slide

**so**

The two expensive parts are gone, and so is data drawn from the policy being trained.

# The On/Off-Policy Scale

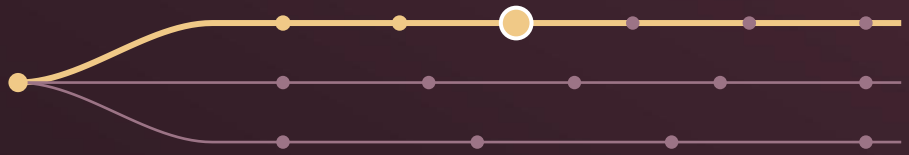



**After the DPO wave the field went back to online RL, without a critic.**

# 02

# A Verifiable Reward

Act I · from imitating reasoning to checking the answer



# Imitate the Chain, or Reward the Answer

**SFT · imitate a reference chain**

$$\mathcal{L}_{\mathrm{SFT}} = -\mathbb{E}_{(x,y^*)\sim\mathcal{D}}\left[\log \pi_\theta(y^* \mid x)\right]$$

`y* comes from the dataset`

- learns how reasoning looks: fluent, confident chains that can still be wrong
- sees only successful chains, so it never practices leaving a dead end
- no exploration: likelihood is maximized on someone else's text
- the ceiling is the teacher

**RL · reward the outcome of its own attempt**

$$\mathcal{J}_{\mathrm{RL}} = \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(\cdot|x)}\left[r(x,y)\right]$$

`y is sampled from the model itself`

- every chain that reaches the right answer is reinforced, detours and self-corrections included
- failed attempts are part of the data
- the ceiling is what the model can reach by sampling (block 8)

**FIG 2.1** Detours and self-corrections are what make RL answers long — the R1 curves in block 4.

# RLVR: A Verifier Instead of a Reward Model

$$r(x, y) = \mathbf{1}\left[\text{verify}(x, y)\right] \in \{0, 1\}$$

### math

the final answer matches the reference after normalization

### code

unit tests pass in a sandbox

### formal proofs

a proof checker such as Lean accepts the proof

`cheap` `reproducible` `no learned model to exploit`

Gameable when the check is incomplete: on rule-induction tasks, RLVR-trained models list per-example labels instead of inducing the rule, and the verifier accepts it.

Name and first open recipe: Tülu 3. One reward for the whole sequence — credit assignment is sidestepped; blocks 3 and 10 come back to it.

**FIG 2.2** The 14 GB reward model of slide 1.2 becomes a program.

# Where a Cheap Check Exists

| cheap to check | expensive or impossible to check |
|---|---|
| math with a numeric answer | summaries and explanations |
| code with unit tests | plans and strategies |
| formal proofs (Lean) | open-ended dialogue |
| puzzles with a unique solution | long-form writing |
| extraction checked against a schema | research reports |

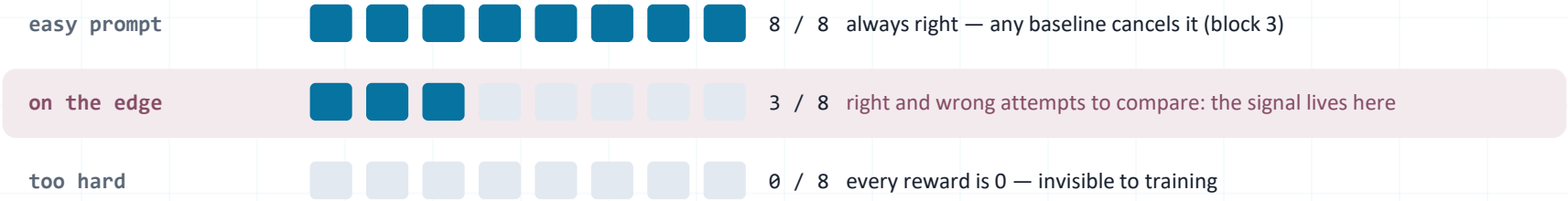
> **The column is decided by whether a cheap check exists: an olympiad problem with a numeric answer goes left, a two-line e-mail goes right.**

FIG 2.3 The right column is where block 12 ends the lecture.

# A Prompt the Model Never Solves Teaches Nothing

$$\nabla_\theta \mathcal{J} = \mathbb{E}_{y\sim\pi_\theta}\left[ r(x,y)\, \nabla_\theta \log \pi_\theta(y \mid x) \right]$$

`r = 0 on every attempt ⇒ the gradient for this prompt is exactly 0`




Hence curriculum and a cold start: keep prompts the model solves sometimes. Block 6 turns this into dynamic sampling, block 8 into adaptive environments.

**FIG 2.4** The middle row is the edge of competence — Act III is built on it.

# 03

# Group Instead of a Critic

Act I · the baseline comes from the samples



# Three Ways to Get a Baseline

PPO

**a critic predicts it**

$$A_t = \mathrm{GAE}\big(r,\, V_\psi(s_t)\big)$$

a second trained model — 112 GB for a 7B policy

→

RLOO · 2024

**the others are the baseline**

$$b_i = \frac{1}{G-1}\sum_{j \neq i} r_j$$

G samples per prompt, no extra model

→

GRPO · 2024

**the group sets zero and scale**

$$A_i = \frac{r_i - \mathrm{mean}(\mathbf{r})}{\mathrm{std}(\mathbf{r})}$$

same idea, normalized by the spread

**Generating the answers happens anyway. Comparing them is cheaper than training a model to guess their average.**

FIG 3.1 The critic was trained to predict the expected reward of a prompt. With several samples per prompt, that expectation can be measured.

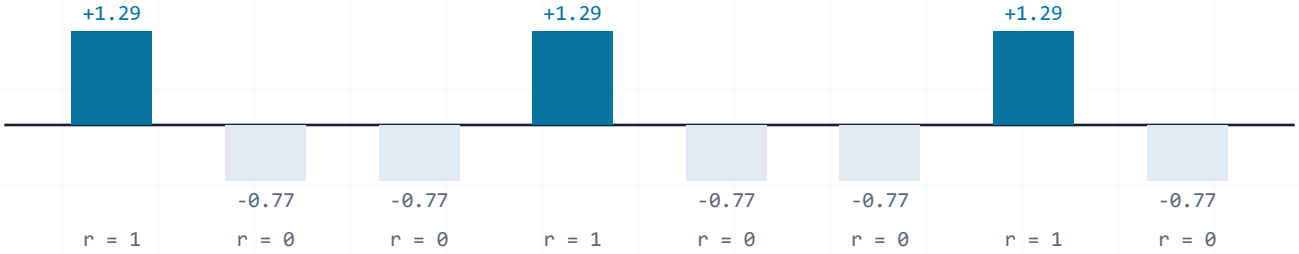
# Group-Relative Advantage

$$\hat{A}_i = \frac{r_i - \text{mean}(r_1, \ldots, r_G)}{\text{std}(r_1, \ldots, r_G)}$$

one prompt,
G = 8 answers, 3 correct

mean = 0.375 → 0

std = 0.48 → unit




Each answer is scored against its siblings: the same reward of 1 is worth more in a group where few succeed.

**FIG 3.2** mean(r) is a Monte-Carlo estimate of V(x) — the number the critic was trained to predict.

# The Full GRPO Objective

$$\mathcal{J}(\theta) = \mathbb{E}\left[\frac{1}{G}\sum_{i=1}^{G} \underbrace{\frac{1}{|o_i|}}_{(1)} \sum_{t=1}^{|o_i|} \min\left(\rho_{i,t}\hat{A}_i,\ \mathrm{clip}(\rho_{i,t}, 1-\varepsilon, 1+\varepsilon)\hat{A}_i\right) - \beta\, \mathbb{D}_{\mathrm{KL}}\left[\pi_\theta \,\|\, \pi_{\mathrm{ref}}\right]\right]$$

$$\hat{A}_i = \frac{r_i - \mathrm{mean}(\mathbf{r})}{\underbrace{\mathrm{std}(\mathbf{r})}_{(2)}}$$

$$\mathbb{D}_{\mathrm{KL}} = \frac{\pi_{\mathrm{ref}}}{\pi_\theta} - \log\frac{\pi_{\mathrm{ref}}}{\pi_\theta} - 1$$

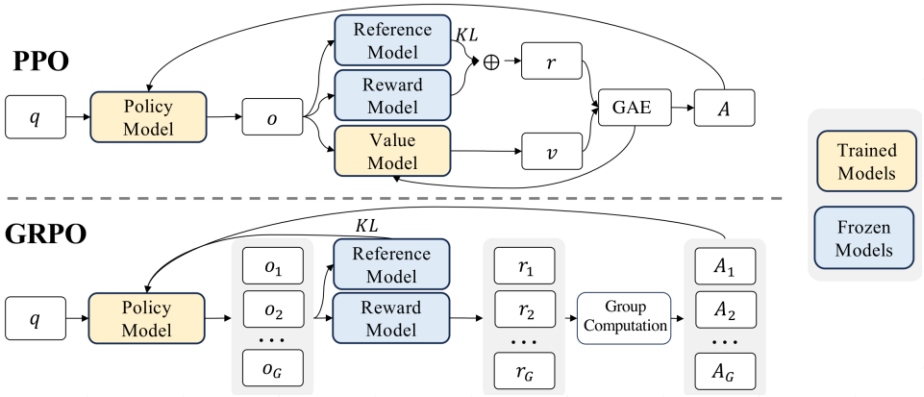
k3 estimator, per token

(1) averaging over the tokens of each answer · (2) division by the group's spread → both come back in block 6 as biases

| | |
|---|---|
| **reward** | correctness + format (answer in a box, reasoning in tags) |
| **in DeepSeekMath** | G = 64 answers per question, β = 0.04, one update per batch |
| **in later RLVR** | the KL term is often weakened or dropped (block 6) |

**FIG 3.3** Everything else is PPO: ratio, clipping, KL to the reference. The advantage is now one number for the whole answer.

# What Leaves Memory




DeepSeekMath, Fig. 4: the value model and GAE are gone; the group computes the advantage

**252 GB**

PPO, slide 1.2

**140 GB**

GRPO removes the trained critic

**126 GB**

RLVR turns the reward model into a program

With the KL term kept. Drop it, as many RLVR recipes do, and the reference leaves too: 112 GB, the actor alone.

**FIG 3.4** Same assumptions as slide 1.2: 7B, bf16 + Adam, no sharding.

# DeepSeekMath-7B: What GRPO Buys

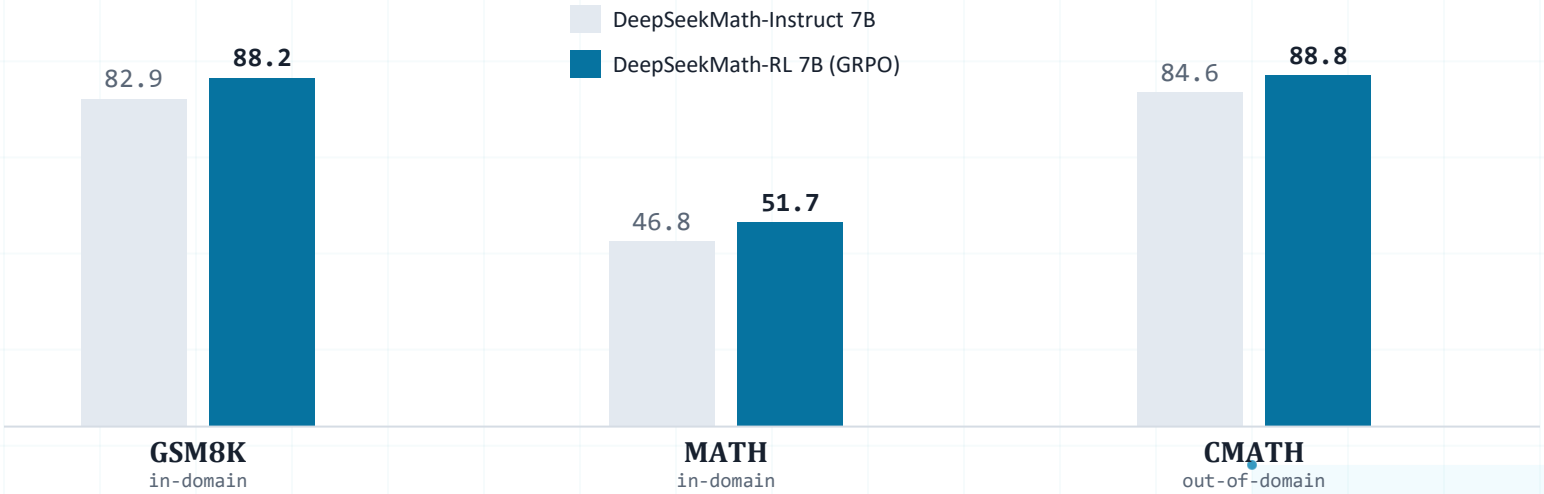



**FIG 3.5** The same 7B model before and after GRPO. The reward here is still a learned reward model — the verifier arrives with R1 in block 4.

Shao et al., 2024 — DeepSeekMath

# Two Blind Spots

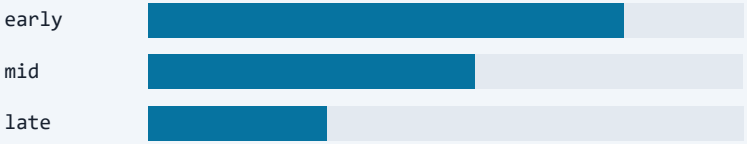
**one answer, |o_i| tokens**

| Let | x | = | 3 | · | wait | , | check | … | = 60 | ■ |
|---|---|---|---|---|---|---|---|---|---|---|
| Â | Â | Â | Â | Â | Â | Â | Â | Â | Â | Â |

Every token gets the same Â_i: the step where the reasoning turned and the filler around it are credited equally.

**dead groups pile up as the model learns**

share of the batch with a gradient · schematic




A group where all 8 agree has Â = 0 (slide 2.4). As accuracy grows, more prompts land at 8 / 8: the batch is generated in full, but fewer of its groups move the weights.

**FIG 3.6** Both get fixed later: credit inside the answer in blocks 6 and 10, dead groups by dynamic sampling in block 6.

**The baseline comes from the samples: answers to one prompt, compared with each other, replace a trained critic.**

KEY IDEA 1 / 6

# 04

# DeepSeek-R1

Act I · long reasoning grown from a reward alone



# R1-Zero: Only a Reward, Straight from the Base

**DeepSeek-V3-Base**

the pretrained base model — no SFT of any kind

→

**GRPO**

16 answers per question, 32 questions per step, 10,400 steps

→

**rule-based reward**

correctness of the final answer + format, with equal weight

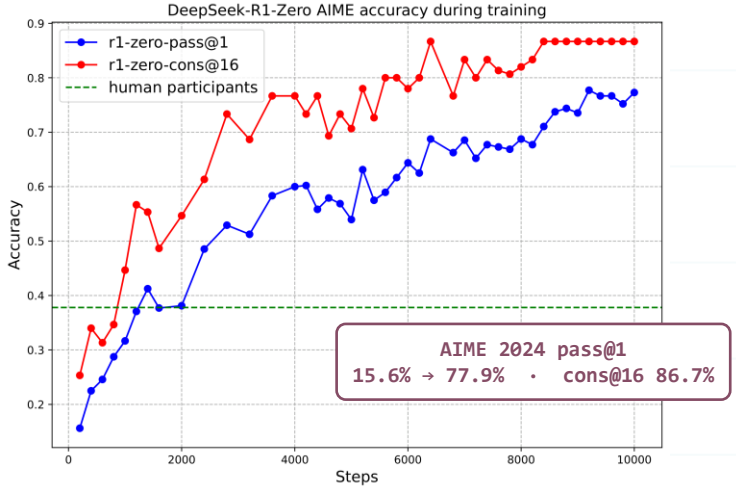
the only instruction: where to put the reasoning

```
<think> reasoning process here </think>   <answer> answer here </answer>
```

**Everything the model writes inside <think> is left to RL. Nobody shows it what a good chain of thought looks like.**

FIG 4.1 No demonstrations, no reward model, no search.

# Two Curves: Accuracy and Length



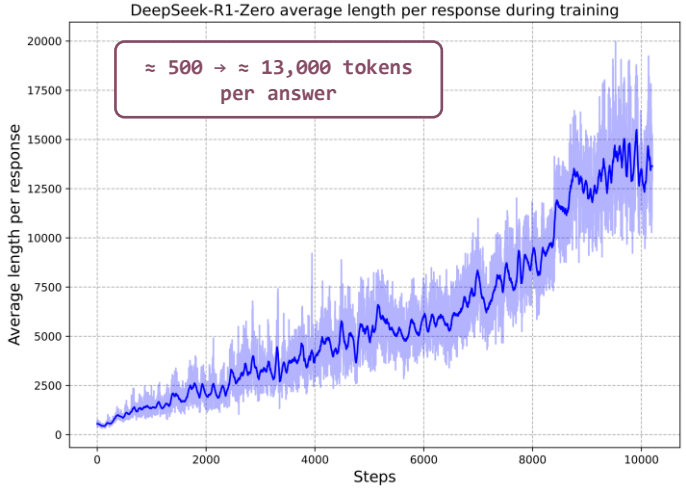




**FIG 4.2** Nothing in the reward asks for length. The jump at step 8.2k is the length cap raised from 32k to 64k tokens.

# The “Aha Moment”

an intermediate R1-Zero checkpoint, mid-solution

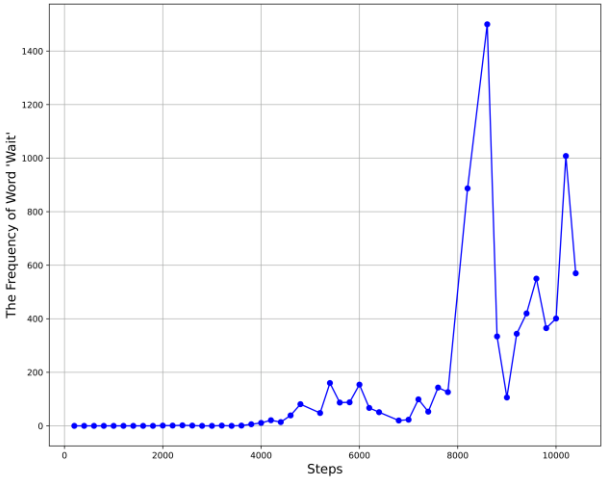
If $a > 1$, solve $\sqrt{a - \sqrt{a + x}} = x$

…squares both sides, isolates the inner root, expands to a quartic in x…

**“Wait, wait. Wait. That's an aha moment I can flag here.”**

…and restarts the derivation step by step.

**re-checking · backtracking · trying another route**




count of the word “wait” per training step

**FIG 4.3** Measured as word frequency. Base models already write these words — block 8 asks what RL actually adds.


Who taught R1-Zero to say “Wait”?
the base model
– Well, of course I know him. He’s me.

# Full R1: Four Stages

**WHAT WAS WRONG WITH R1-ZERO**

**unreadable**

mixes English and Chinese inside one chain of thought (the base saw mostly both); the summary holds only the final answer

**reasoning-only · R1-Zero → R1**

rule rewards cover only math, code and logic
IF-Eval 46.6 → 83.3 · AlpacaEval 2.0 24.7 → 87.6 · ArenaHard 53.6 → 92.3

**1 · cold-start SFT**

thousands of readable long chains of thought

`fixes: readability`

→

**2 · reasoning RL**

GRPO, rule rewards + a language-consistency reward

`fixes: mixed languages`

→

**3 · rejection sampling + SFT**

~600k reasoning + ~200k other samples, fine-tuned from the base

`fixes: writing, general tasks`

→

**4 · RL for all scenarios**

rule rewards for reasoning, reward models for helpfulness and harmlessness

`fixes: preferences, safety`

**Pure RL is the demonstration. The released recipe alternates SFT and RL, and each stage patches what the previous one left.**

FIG 4.4 R1 vs. o1-1217 on AIME 2024, pass@1: 79.8 vs. 79.2. R1-Zero still beats R1 on GPQA Diamond (75.8 vs. 71.5) and CNMO 2024 (88.1 vs. 78.8).

# Distillation Beats RL on a Smaller Base

| | AIME 2024 | MATH-500 | LiveCodeBench |
|---|---|---|---|
| Qwen2.5-32B + large-scale RL | 47.0 | 91.6 | 55.0 |
| **Qwen2.5-32B + SFT on 800k R1 samples** | **72.6** | **94.3** | **62.1** |

smaller distilled models on AIME 2024: 14B — 69.7 · 7B — 55.5

> **+25.6 points on AIME from copying R1's traces instead of running the RL recipe on the same base. Reasoning patterns transfer more cheaply than they are rediscovered.**

FIG 4.5 Block 11: frontier models of 2026 distill RL-trained specialists into one model.

**A check of the final answer is enough: long reasoning grows from the reward alone, and distillation carries it to smaller models.**

KEY IDEA 2 / 6

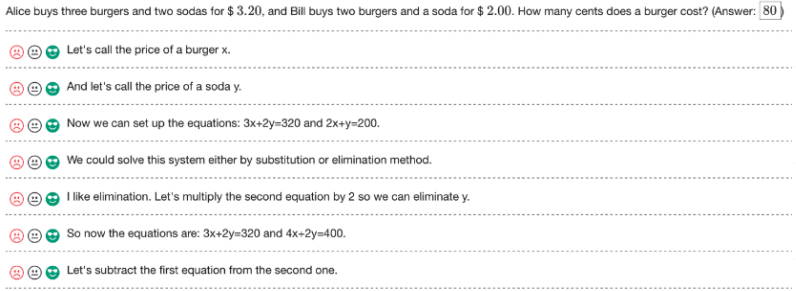
# 05

# Step Rewards: Tried, Dropped

Act I · a step back to 2023: why not reward every step?



# Let's Verify Step by Step: Humans Label Every Step




**PRM800K: 800K step labels · 75K solutions · 12K problems**

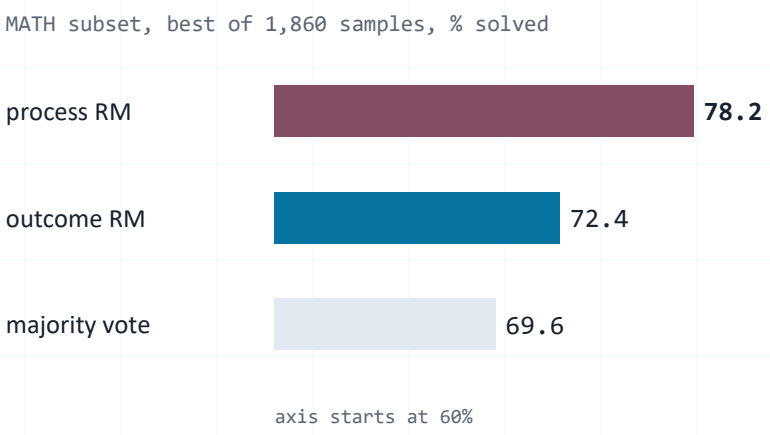
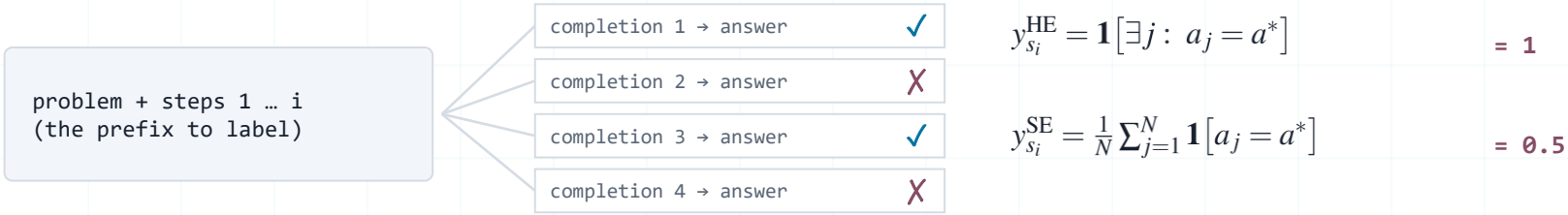



**FIG 5.1** Used to rerank finished answers (best-of-N), with no RL on top.

# Math-Shepherd: Label a Step by Finishing It




**Step-level PPO with these labels, Mistral-7B: GSM8K 77.9 → 84.1 · MATH 28.6 → 33.0**

No human labels: a completer model rolls each prefix forward, and the fraction of correct endings becomes the step's label.

**FIG 5.2** Slide 1.3 said: to judge a prefix you have to finish it. Math-Shepherd does exactly that, N times per step.

# Why R1 Dropped Step Rewards

### process reward model

- a “step” is hard to define in free-form reasoning
- auto-labels are noisy: completions reach the right answer from a wrong step and the wrong answer from a right one
- a learned PRM gets hacked, and retraining it adds a second loop

### Monte Carlo tree search

- branching over tokens is far wider than over moves in chess
- capping the branching gets the search stuck in local optima
- everything hinges on a fine-grained value model that is hard to train

> **Structure with a learned judge cost more than extra samples. R1 kept the outcome reward and spent the compute on rollouts.**

FIG 5.3 The R1 authors add that this does not prove these approaches cannot work. PRMs still help to rerank answers at inference.

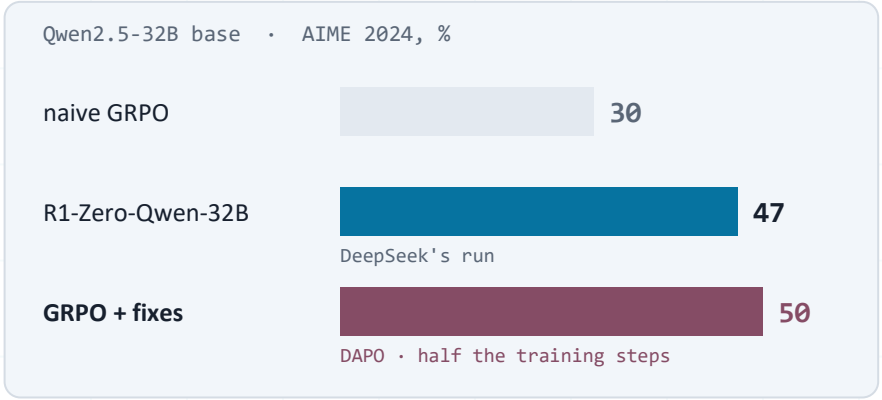
# 06

# Biases in the GRPO Loss

Act II · same base, same algorithm: 30 or 50 on AIME



# One Base Model: 30, 47 or 50 on AIME




**where the points hide**

| | | |
|---|---|---|
| $1/\lvert o_i \rvert$ | each answer averaged over its own length | 6.3–6.4 |
| ÷ std(r) | each group rescaled by its own spread | 6.3–6.4 |
| batch | dead groups and cut-off answers in the update | 6.5 |
| clip | the symmetric clip barely lets rare tokens grow | 6.6–6.8 |

**Same base, same family of algorithms: twenty AIME points sit in the details of the loss and the batch. This block goes through them.**

FIG 6.1 Naive GRPO and GRPO + fixes (DAPO) are ByteDance runs on the same data (DAPO-Math-17K); R1-Zero-Qwen-32B is DeepSeek's run on math, code and STEM, from the R1 paper.

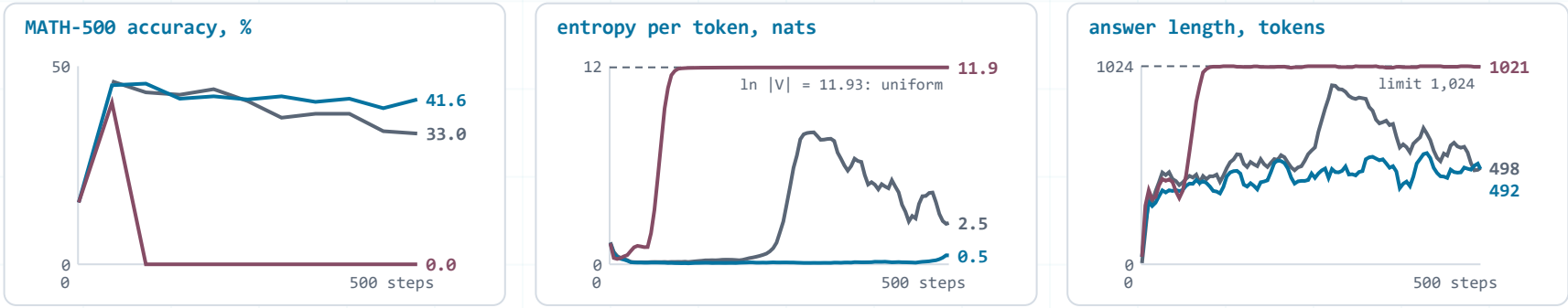
# What a Failing GRPO Run Looks Like

Qwen3-0.6B base · 12k MATH problems · 8 answers per problem, ≤ 1,024 tokens · MATH-500 every 50 steps

plain: advantage × log p, no ratio, no KL

+ PPO ratio (ε = 10), 2 updates per step

+ KL term as a loss, β = 0.001




**Accuracy peaks at step 50 and decays as entropy explodes. With the KL term, entropy reaches ln |V|: the model emits uniform noise.**

FIG 6.2 One seed per variant; the variants differ in more than one setting, so the panels show symptoms, not causes. Curves: 25-step moving average.

# Dr. GRPO: Two Strike-Throughs

$$\mathcal{J}(\theta) = \mathbb{E}\left[\frac{1}{G}\sum_{i=1}^{G}\cancel{\frac{1}{|o_i|}}\sum_{t=1}^{|o_i|}\min\left(\rho_{i,t}\hat{A}_i,\ \text{clip}(\rho_{i,t}, 1-\varepsilon, 1+\varepsilon)\hat{A}_i\right)\right] \qquad \hat{A}_i = \frac{r_i - \text{mean}(\mathbf{r})}{\cancel{\text{std}(\mathbf{r})}}$$

struck: $1/|o_i|$ — averaging inside each answer · $\text{std}(\mathbf{r})$ — rescaling inside each group

**what is left**

REINFORCE with the group mean as a baseline; the code divides by a constant, e.g. the generation budget.

**the same 1/|o| in open PPO code**

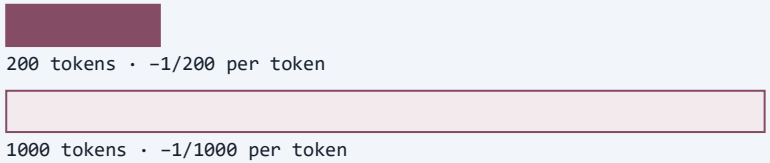
trl · OpenRLHF · verl · SimpleRL-Zero · Open-Reasoner-Zero: the masked mean divides by response length.

**Without the two factors, wrong answers stop growing longer and accuracy holds.**

FIG 6.3 Length can be an artifact of the loss (this slide), a penalty in the reward, an overlong-reward rule (6.5) or a trained budget knob (block 11).

# Why the Two Factors Bias the Update

**$1/|o_i|$: one wrong answer, $\hat{A} = -1$**




A long wrong answer is punished 5× less per token, and a short correct one is rewarded more: wrong answers drift longer.

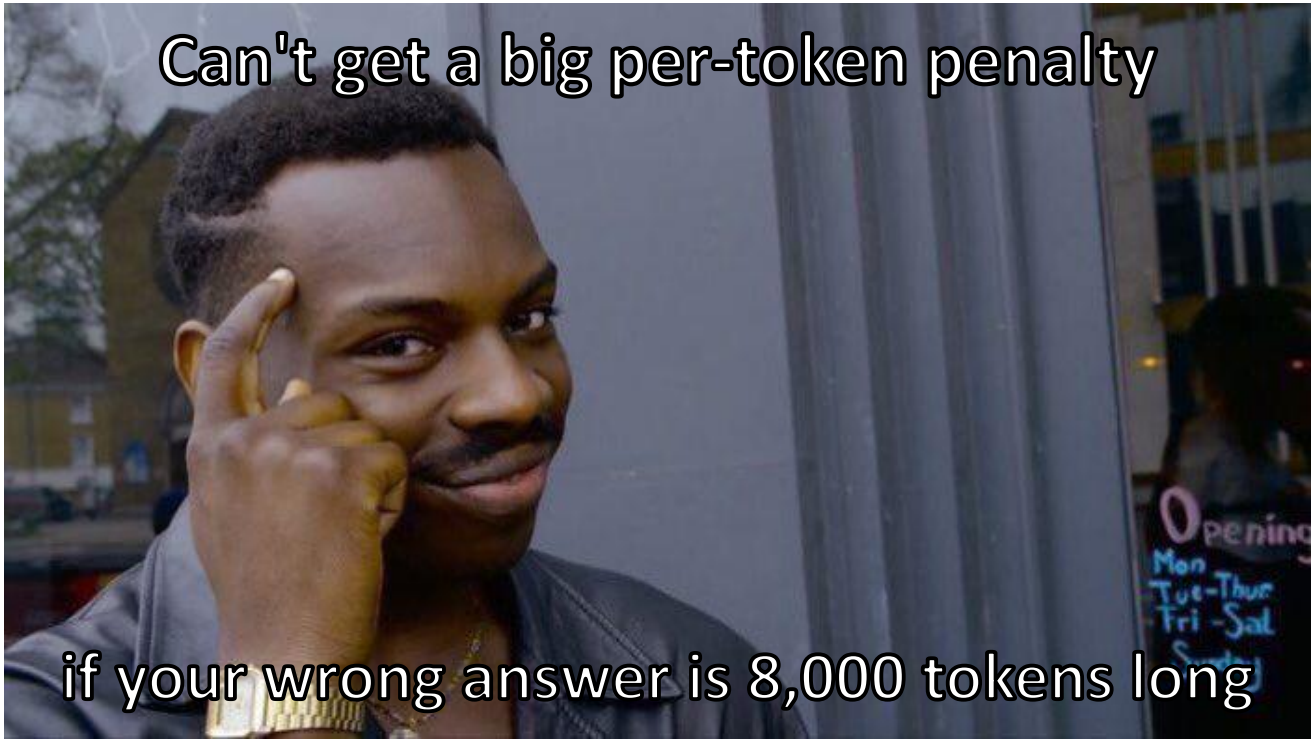
**÷ std(r): weight of a prompt, G = 8**



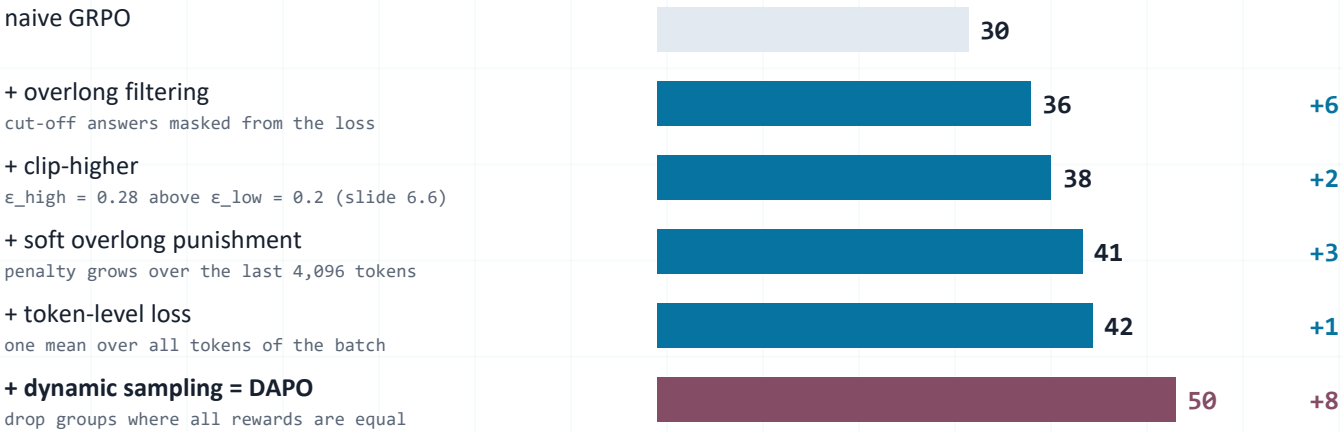

Near-unanimous prompts, too easy or too hard, get 1.5× the weight of one the model solves half the time.

**Both biases come from normalization, not from the reward: they change which answers and which prompts the update listens to.**

FIG 6.4 Toy numbers. Mechanism: Liu et al., Sec. 3.1.


Can't get a big per-token penalty
if your wrong answer is 8,000 tokens long

# DAPO: From 30 to 50, One Fix at a Time



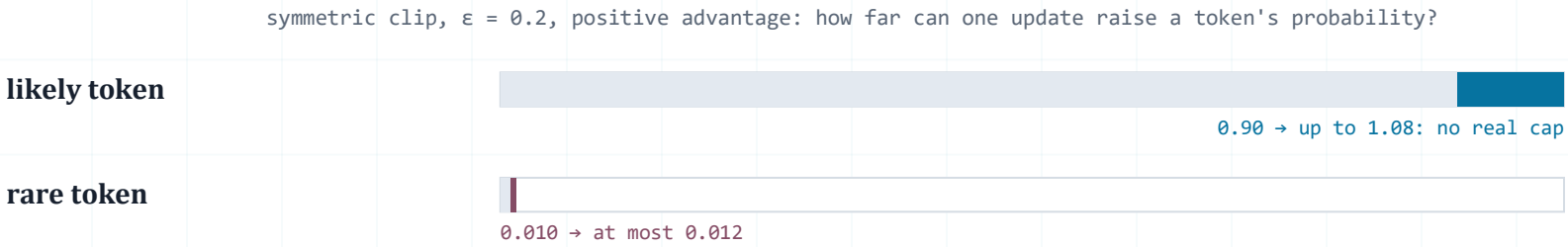

**the run**

- Qwen2.5-32B base
- DAPO-Math-17K
- 512 prompts × 16
- 16 updates per rollout
- no KL term
- max 20,480 tokens
- DeepSeek's run: 47

**FIG 6.5** The 30 and 50 of slide 6.1. Progressive ablation: each row adds a technique to all rows above, so a row's gain depends on the authors' order. AIME 2024, avg@32.

# Clip-Higher: The Upper Bound Throttles Rare Tokens

$$\text{clip}\left(\rho_{i,t},\ 1-\varepsilon_{\text{low}},\ 1+\varepsilon_{\text{high}}\right), \qquad \varepsilon_{\text{low}} = 0.2, \quad \varepsilon_{\text{high}} = 0.28$$

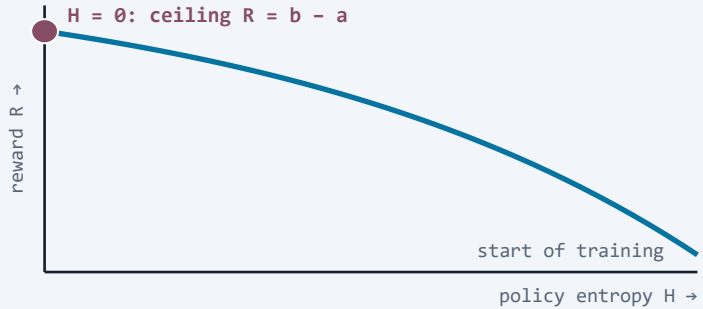



**A symmetric clip lets likely tokens grow and caps the rare ones, so exploration dies first. Raising only the upper bound fixes that.**

FIG 6.6 The tokens that hit the upper clip have mean probability below 0.2 (DAPO, Fig. 3a). Block 7: CISPO keeps their gradient instead of dropping it.

# Entropy Is Spent to Buy Reward

$$R = -a\,e^{H} + b$$




**how fast it is spent**

first 200 of 2,400 steps: 73% of the entropy drop and 76% of the gain. First 800: 94% and 93%.

**why it falls**

$$\Delta H \approx -\eta\,\mathrm{Cov}_{a\sim\pi}\big(\log \pi(a\,|\,s),\, A(s,a)\big)$$

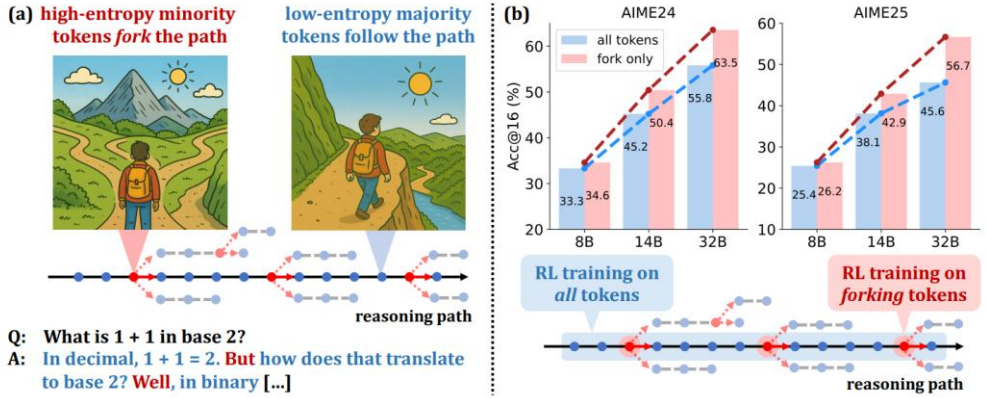
a likely token with a positive advantage lowers entropy; a rare one raises it

**the fix**

Clip-Cov / KL-Cov hold back the few highest-covariance tokens. KL-Cov: +2.0 (7B) and +6.4 (32B) points over GRPO, mean of 7 benchmarks.

**FIG 6.7** Fitted on 11 base models from 4 families, 0.5B–32B, 2,400-step GRPO runs without entropy control.

# Learning Happens at the Forks




### the 20% that fork

Qwen3-8B on AIME, $10^6$ tokens: half have entropy below 0.01, only 20% exceed 0.672 — wait, however, thus, suppose, assume.

### fork only vs all tokens

DAPO, Acc@16. 32B: +7.7 on AIME'24, +11.1 on AIME'25. Training on the other 80% alone makes the model markedly worse.

**The fifth of tokens that choose the path carries the learning, and the effect grows with the model.**

FIG 6.8 Llama-3.1-8B needs an SFT cold start before DAPO trains at all; there the effect is present but, per the authors, less convincing.

# 07

# Off-Policy in Practice

Act II · the samples come from a policy that is no longer there



# Three Ratios Behind One Gradient

one reward per answer · one ratio per token · and the samples come from a policy that is no longer the one being updated

**1 · the policy moved**

$$\frac{\pi_\theta(o_t)}{\pi_{\theta_{\text{old}}}(o_t)}$$

several gradient steps per batch of rollouts (DAPO: 16)

→ GSPO, CISPO · 7.2

**2 · the engine is not the trainer**

$$\frac{\pi^{\text{train}}_{\theta_{\text{old}}}(o_t)}{\pi^{\text{infer}}_{\theta_{\text{old}}}(o_t)}$$

same weights, other kernels: rounding, MoE routing, the top-p cut

→ R3, FP16 · 7.3

**3 · the rollout is old**

$$\frac{\pi_\theta(o_t)}{\pi^{\text{infer}}_{\theta_{k-d}}(o_t)}$$

asynchronous generation: the trajectory began d versions ago

→ async engines · 7.4

**At scale every RL step is off-policy. A recipe decides how far, and against which probabilities the gap is measured.**

FIG 7.1 Slide 1.5 put PPO and GRPO at the on-policy end of the scale. These three ratios are why they never quite reach it.

# GSPO and CISPO: Where the Correction Goes

**GSPO · one ratio per answer**

$$s_i = \left( \frac{\pi_\theta(y_i \mid x)}{\pi_{\theta_{\text{old}}}(y_i \mid x)} \right)^{1/|y_i|}$$

| | |
|---|---|
| clip | $3 \cdot 10^{-4}$ / $4 \cdot 10^{-4}$ on the answer, against 0.2 / 0.27 per token in GRPO |
| MoE | ~10% of active experts change after one update; GRPO needed Routing Replay, GSPO does not |
| used in | the latest Qwen3 models |

**CISPO · clip the weight, keep the token**

$$\text{sg}\left(\text{clip}(\rho_{i,t}, 1-\varepsilon_{\text{low}}, 1+\varepsilon_{\text{high}})\right) \hat{A}_{i,t} \log \pi_\theta(o_{i,t})$$

| | |
|---|---|
| which tokens | rare forks (However, Recheck, Wait) keep their gradient after the first of 16 updates |
| result | AIME 2024: DAPO's score in half the steps |
| at scale | MiniMax-M1: the whole RL run on 512 H800 in 3 weeks, ≈ $0.53M |

**The reward belongs to the answer and the gradient to each token: GSPO moves the ratio to the answer, CISPO caps a token's weight and keeps its gradient.**

FIG 7.2 GSPO: Qwen3-30B-A3B cold start, GRPO with tuned clip 0.2 / 0.27 as baseline. CISPO: Qwen2.5-32B base, same data as DAPO.

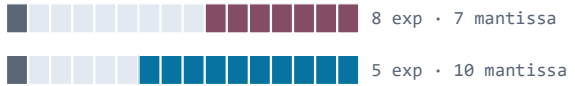
# Same Weights, Different Probabilities

### MoE routing · R3




| | |
|---|---|
| ~10% · 94% | router choices that differ · tokens that differ in ≥ 1 layer |
| fix | cache the rollout's routing masks and reuse them in training |
| 62.2 → 71.8 | average score; GRPO's collapse at step 60 is gone |

### rounding · FP16

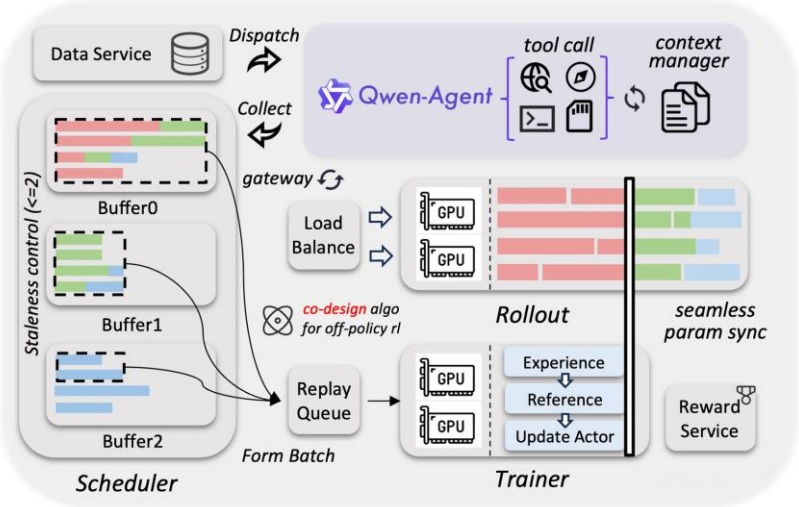



| | |
|---|---|
| 8× finer | FP16 rounds 8× finer; RL weights no longer need BF16's range |
| 73–84% → ✕ | GRPO in BF16 peaks and collapses; in FP16 it trains on |
| one format | Jet-RL: one FP8 flow for both · Kimi K3: one MXFP4 scheme |

**The trainer has to compute the probabilities the engine sampled from: same experts, same rounding.**

FIG 7.3 R3: Qwen3-30B-A3B, rollouts in SGLang, training in Megatron. FP16: 1,460 MATH problems that R1-Distill-Qwen-1.5B solves 20–80% of the time.

# The Engine Is Part of the Training Strategy




**what the post lists · where we met it**

| | |
|---|---|
| fully disaggregated training and inference | async |
| FP8 end-to-end training | 7.3 |
| rollout router replay | 7.3 |
| multi-turn rollout locking | async |
| co-design bounds gradient staleness | 7.1–7.2 |
| speculative decoding | lecture 12 |

async elsewhere: Kimi K3 lets one trajectory span several policy versions; GLM-5 passes the engine's token IDs, never re-tokenized text

**3×–5×** end-to-end speedup of the whole RL loop

**THE LESSON**

The gradient is correct only for the numbers the engine really produced; half the recipe keeps them equal.

**FIG 7.4** Figure from the Qwen3.5 post. It gives no per-technique ablation, only the list and the total speedup.

# Act II on One Page: Symptom, Fix, Source

| problem | what you see | fix | source | slide |
|---|---|---|---|---|
| **no signal** | all rewards in a group equal | drop the group, sample more | DAPO | 6.5 |
| **length bias** | long wrong answers punished less per token | token-level mean or a constant divisor | Dr. GRPO, DAPO | 6.3–6.4 |
| **difficulty bias** | near-unanimous groups weigh more | no division by std(r) | Dr. GRPO | 6.4 |
| **cut-off answers** | truncation scored as a wrong answer | mask them; soft penalty near the limit | DAPO | 6.5 |
| **exploration dies** | rare tokens capped, entropy collapses | clip-higher; Clip-Cov / KL-Cov; train on forks | DAPO, Cui, Wang | 6.6–6.8 |
| **noisy ratios** | one ratio per token on a 20k-token answer | one ratio per answer, or clip the weight only | GSPO, CISPO | 7.2 |
| **engine ≠ trainer** | same weights, other probabilities | replay routing and top-p mask; FP16; truncated IS | R3, Qi, Yao, V3.2 | 7.3 |
| **stale rollouts** | trajectory spans policy versions | bound staleness; mask far-off negative answers | Kimi K3, GLM-5, V3.2 | 7.4 |
| **KL term** | biased gradient; holds the model back | drop it for math, or k3 with an IS ratio | DAPO, Dr. GRPO, V3.2 | 3.3, 6.2 |

> **The loss is fixed and the samples are honest. Act III asks what this machine actually teaches.**

**FIG 7.5** Each fix is one option, not a fixed recipe: DeepSeek-V3.2, for instance, keeps a weak KL term and drops the std division.

**At scale every RL step is off-policy: the gradient is right only for what the engine actually sampled.**

KEY IDEA 3 / 6



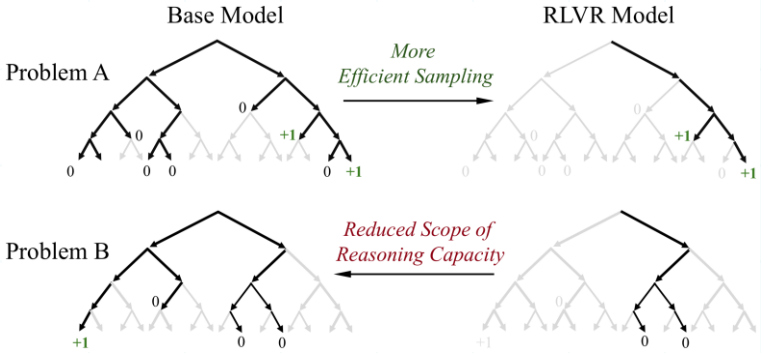
# 08

# Where the Gains Come From

Act III · what RL adds to the base, and the data it needs



# pass@1 Goes Up, pass@256 Does Not




black: sampled paths · +1: a correct answer

**as training goes on**

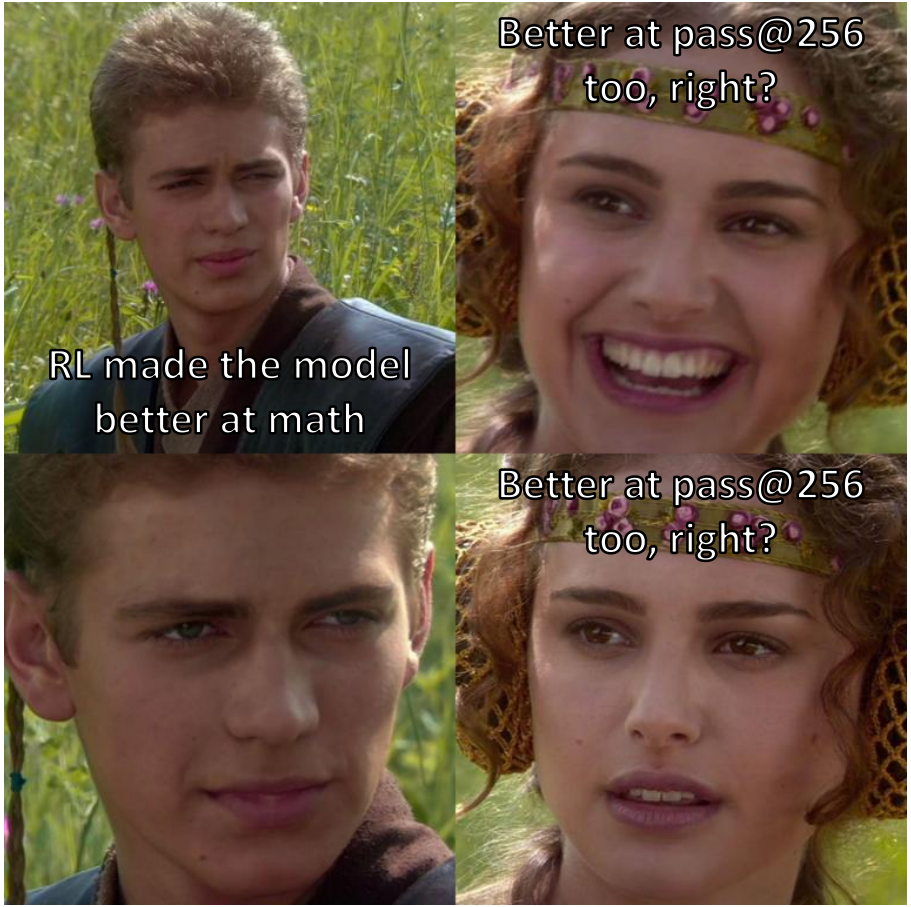
| step | pass@1 | pass@256 |
|---|---|---|
| 0 · base | 9.9 | 67.2 |
| 150 | 26.1 | 66.3 |
| 300 | 33.6 | 65.3 |
| 450 | 42.5 | 64.3 |

**one small model · Qwen3-0.6B, MATH-500, %**

| | |
|---|---|
| base, majority of 10 sampled CoT answers, 4.7× the time | 52.0 |
| trained thinking variant, one answer | 48.2 |
| base after 50 GRPO steps, one answer | 47.4 |

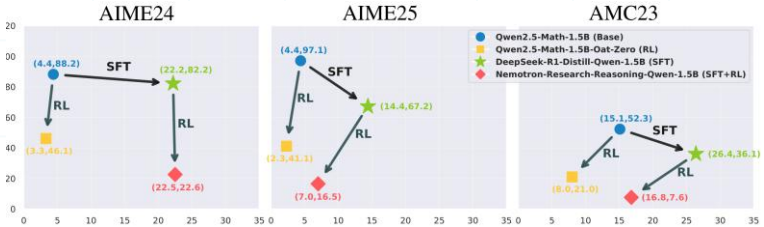
**RLVR makes the model reliable at what the base could already sample; it rarely adds problems the base never solves.**

**FIG 8.1** Qwen2.5-7B base, GRPO on Omni-MATH, measured on the training problems. Six RLVR algorithms land within 2.2 points of each other at pass@256.


RL made the model better at math
Better at pass@256 too, right?
Better at pass@256 too, right?

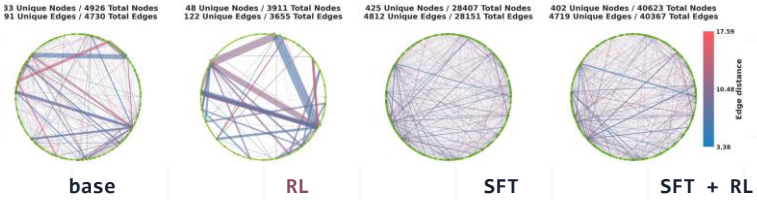
# RL Squeezes, SFT Expands

distinct solutions: correct (x) vs incorrect (y)




| | |
|---|---|
| RL | incorrect 88 → 46, correct 4.4 → 3.3 |
| SFT | correct 4.4 → 22 |
| SFT, then RL | 22.5 correct, 22.6 incorrect |

reasoning graph of one problem, edges by frequency




RL routes the solutions through a few hub steps: visit frequency, degree and centrality fall off 2.5× faster with rank. SFT spreads them out: the decay drops to about a third.

**RL prunes wrong paths and some right ones; SFT adds new right paths. Hence the recipe: SFT first, then RL.**

FIG 8.2 Left: 1.5B models, 256 samples per problem, solutions clustered by text similarity. Right: AIME24 problem 1, 256 responses per 7B model; nodes are clusters of sentences.

# The Base Knows the Moves, RL Teaches When

**four moves, counted in the trace text**

- verification
- backtracking
- subgoal setting
- backward chaining

Countdown accuracy after the same RL




priming works even with wrong answers that show the moves

**a base model, steered at the right tokens**

Steering vectors for each move are switched on in the base model wherever the thinking model would make that move. The weights stay as they are.

| | |
|---|---|
| **91%** | of the gap to the thinking model, at most |
| **12%** | of tokens steered |
| **0** | weight updates |

**A long trace is built from moves the base model already makes. RL mostly teaches when to make them.**

**FIG 8.3** Left: Countdown, the same RL for both 3B models. Right: GSM8K and MATH500, base models from 1.5B to 32B paired with four thinking models.

# Gains Appear Only at the Edge of Competence

**difficulty = number of operations in the problem**

| pre-training range | edge: op 11–14 | beyond: op 15–20 |
|---|---|---|
| pass@1 ↑ · pass@128 flat: RL only sharpens | pass@128 ↑, up to +42% | base near zero; gains only via edge training |

op 2 op 6 op 10 op 14 op 20

**RL data, same budget**

| | |
|---|---|
| op 7–10 · inside | no new problems solved |
| op 11–14 · the edge | pass@128 grows on both OOD ranges |
| op 17–20 · too hard | reward plateaus: too few successes |

**what pre-training must plant**

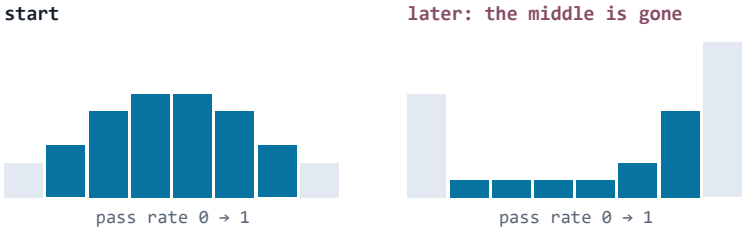
≥ 1% of a new context in pre-training: RL carries the skill there, up to +60% pass@128; 0.1%: nothing. Mid-training + RL beats RL alone by +10.8%.

> **RL widens what the base can solve only at the edge of competence, and only where pre-training planted a seed.**

**FIG 8.4** Controlled lab: a 100M Qwen2.5-style model pre-trained on 10B tokens of synthetic problems with 2–10 operations; GRPO on 200K problems from one range.

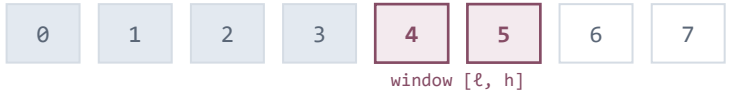
# The Edge Moves, So the Data Must Move Too

**a static set: pass rate per prompt**

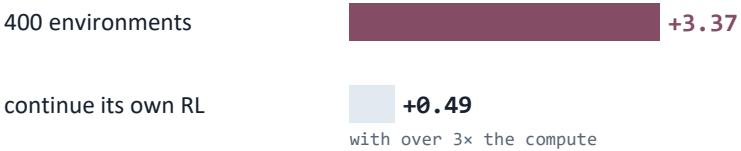



Grey bins give Â = 0. Qwen-4B-Instruct stalls after ~300 steps on data that fed a 1.5B model for 2,000.

**RLVE: each environment tracks the edge**




sorting: difficulty = array length. When accuracy at level h passes a threshold, h goes up by one.




**The edge moves with the model. Environments that raise the bar keep the training data on it.**

FIG 8.5 Left: schematic histograms; the saturation numbers are from Golden Goose. Right: RLVE from ProRL-1.5B-v2, average over six reasoning benchmarks.

# Out of Verifiable Data? Make It from Text

**any text**

a textbook, a solution, a web page

→

**mask a key step**

an LLM picks the reasoning span

→

**add ≥ 10 distractors**

plausible, similar, wrong

→

**a verifiable MCQ**

the masked span is the answer

**continued RL after saturation, average points**




**a domain with no RLVR data**

180K cybersecurity tasks from FineWeb pages: +4.44 points on three benchmarks after 100 RL steps.

**Any reasoning-rich text becomes a check, so the supply of problems stops being the limit.**

FIG 8.6 Qwen-4B-Instruct after it saturates on the ProRL data; averages over its math, code and STEM benchmarks.

**RL sharpens what the base can already do and widens it only at the edge of competence.**

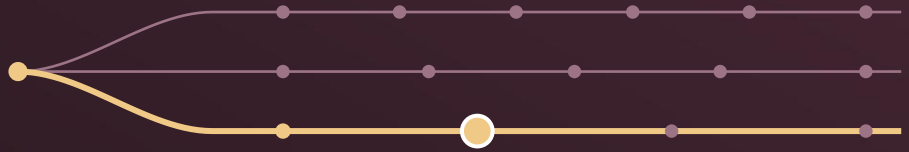
KEY IDEA 4 / 6



# 09

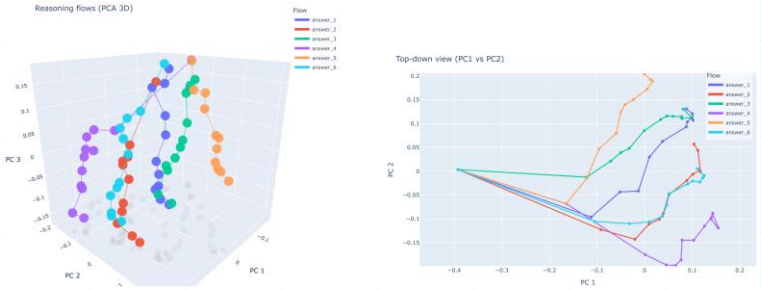
# Inside a Reasoning Trace

Act III · what the long chain is made of, and what it costs



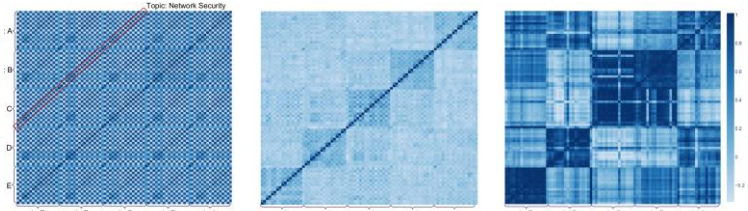
# The Trace Is a Path, and Logic Steers It

**hidden state after each step, projected by PCA**




Each step extends the prefix; the representation moves. Position, velocity and curvature of that path are measured.

**similarity of paths across 5 logics × topics**



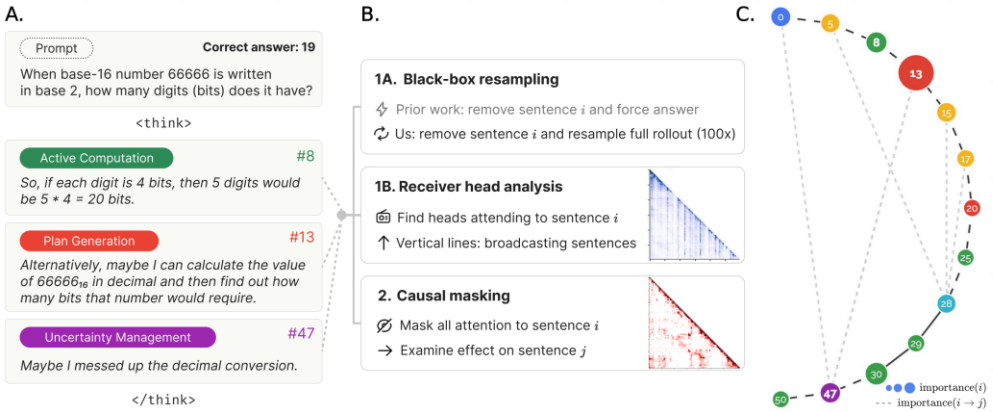

(a) Position Similarity. (b) Velocity Similarity. (c) Curvature Similarity.

Position groups by topic. Velocity and curvature group by logical form. Shuffle the steps, and that grouping disappears.

> **Where the state is says what the text is about; how it moves says which logic the argument follows.**

**FIG 9.1** Left: six answers to one MATH500 problem. Right: Qwen3-0.6B on synthetic natural-deduction chains, logic templates A–E × topics × languages.

# A Few Sentences Steer the Whole Solution




what anchors the trace

plan generation

uncertainty management

highest importance

computation and fact retrieval follow the plan

Importance = how much the answer distribution shifts when the sentence is resampled, 100 rollouts each.

**Planning and backtracking sentences anchor the rest of the trace; the computation in between mostly follows them.**

FIG 9.2 R1-Distill-Qwen-14B and R1-Distill-Llama-8B on 20 MATH problems they solve 25–75% of the time; eight sentence categories.

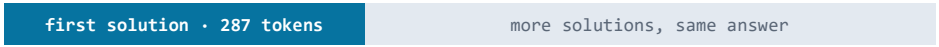
# The Answer Comes Early, the Rest Confirms It

**a long response is several solutions in a row**

**>92%** of correct responses already have the answer in the first solution

**<60%** of the tokens go into that first solution

QwQ-32B-Preview on ASDIV, average response




solutions per response on MATH500




**reflection after the first answer**

**16.8–47.8%** of the tokens

**+1.4–3.5** accuracy points it buys

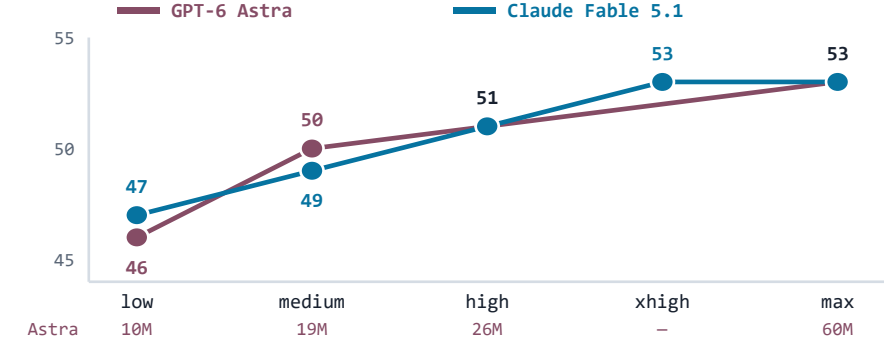
**rarely** does it change the answer

Training on traces with more reflection improves the first answer, more than the ability to fix it later.

**The first solution usually holds the answer; later rounds mostly re-solve the problem and agree.**

**FIG 9.3** Left: QwQ-32B-Preview and DeepSeek-R1 on ASDIV, GSM8K and MATH500, responses with a correct answer. Right: 8 reasoning models on 5 math datasets.

# More Effort Still Buys Accuracy




**what each Astra step costs**

| step | points | tokens |
|---|---|---|
| low → medium | +4 | +9M |
| medium → high | +1 | +7M |
| high → max | +2 | +34M |

Every Astra level sits on the Pareto frontier of index vs tokens. Claude levels off at the top: xhigh and max score the same.

**Across effort levels, longer reasoning buys accuracy, and the top step costs the most tokens per point.**

FIG 9.4 Artificial Analysis Intelligence Index v4.3.2, September 2026. Tokens: output tokens to run the whole index; per-level tokens for Claude are not published.

# Reasoning Without Writing It Down

**text CoT**

### every step is a token

One forward pass per token, and each step squeezed through one of ~150K discrete choices.

**latent CoT**

### the hidden state is the step

The last hidden state goes back in as the next input: a continuous thought, never decoded into words.

**how to teach it**

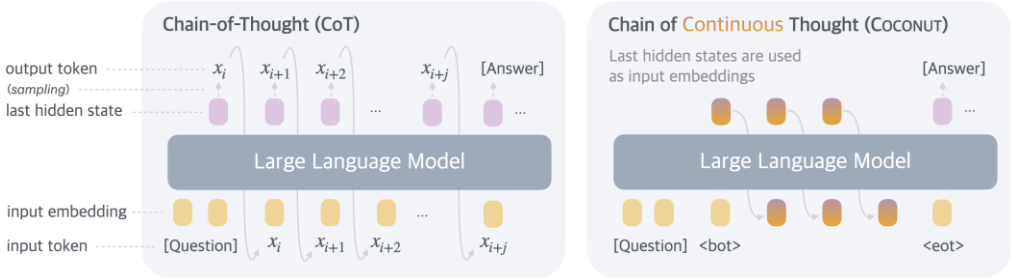
### the text trace becomes a teacher

Coconut swaps text steps for thoughts by curriculum, CODI distills one point, KaVa the whole compressed cache.

**Slide 9.3: much of a trace re-checks an answer already found. Latent reasoning asks whether the steps need to be words at all.**

FIG 9.5 Three papers, one line: Coconut (2024) → CODI (2025) → KaVa (2025). All three train small models (GPT-2 to 3B) on grade-school math.

# Coconut: Feed the Hidden State Back




**how it trains**

Curriculum: stage k replaces the first k text steps with c·k thoughts. Without it: 14.4 on GSM8k.

**GSM8k, GPT-2**

text CoT 42.9 · Coconut 34.1 · No-CoT 16.5, at 8 tokens instead of 25

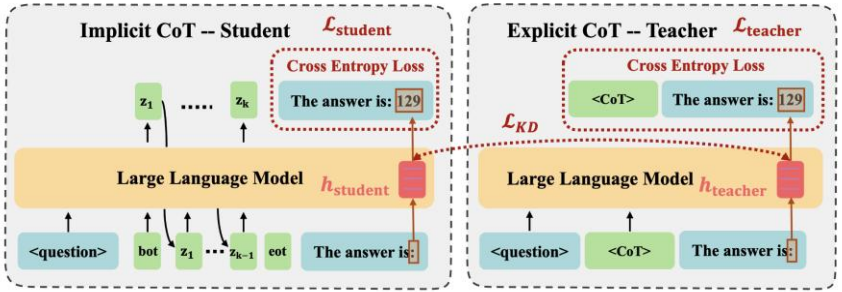
**ProsQA**

text CoT 77.5 · Coconut 97.0: a thought holds several next steps at once, like breadth-first search

> **Continuous thoughts can keep several paths open and win on search; on math they still trail the written chain.**

**FIG 9.6** GPT-2, GSM8k-Aug; ProsQA is a synthetic graph-search logic task. Llama-3-8B on GSM8k: 42.2 → 43.6 over No-CoT.

# CODI: The Model Distills Its Own Chain




**the trick**

No curriculum: match the student's hidden state to the teacher's at one point, the token before the answer.

**GPT-2**

CODI 43.7 against text CoT 44.1 (99%); Coconut 34.1

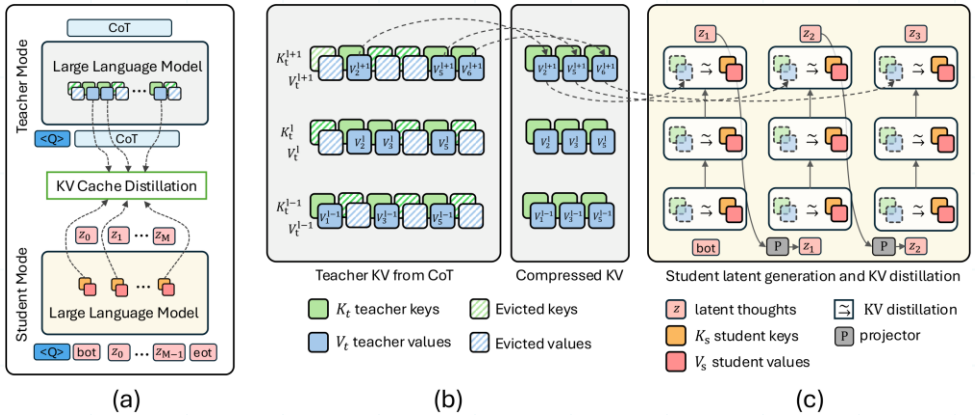
**LLaMA-1B and cost**

55.6, 90% of text CoT. 3.1× shorter, 2.7× faster at batch 1

**Self-distillation instead of a curriculum: the first latent method to match a written chain, on GPT-2.**

**FIG 9.7** GSM8k-Aug, accuracy %. One model, two passes: teacher with the text CoT, student with latent thoughts; LoRA on GPT-2 and LLaMA-3.2-1B.

# KaVa: Distill the Compressed Cache




**the idea**

Compress the teacher's CoT cache to the latent budget, then match the student's keys and values at every layer and step.

**accuracy, AUG · NL**

KaVa 56.5 · 55.7 · CODI 53.9 · 50.1 · text CoT 63.4 · 53.2

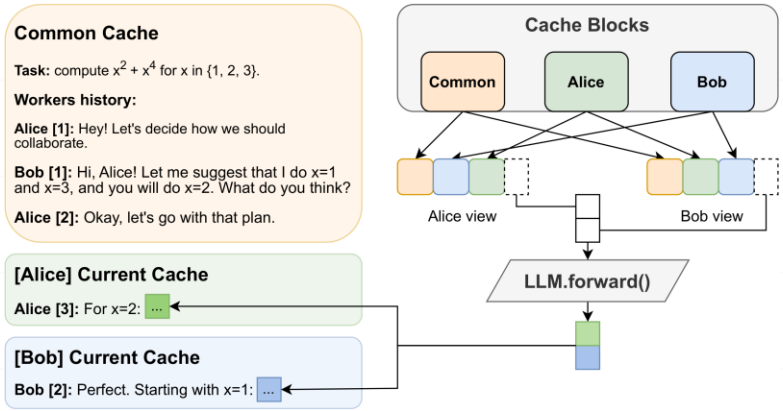
**cost**

6.9 forward passes against 31.9 for text CoT: −78%

**Supervise the whole latent trajectory, not one point: the gap to the written chain shrinks, at a fifth of the forward passes.**

**FIG 9.8** Llama-3.2-1B, GSM8k, accuracy %; AUG: equation-only traces, NL: natural-language traces. Forward passes: trace and answer, average.

# Several Workers, One Shared Cache



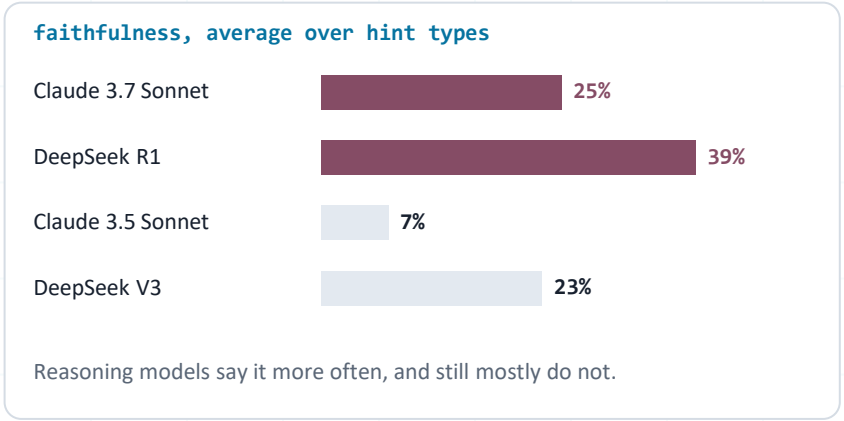

what the workers do on their own

- agree how to split the problem in the first few steps
- notice duplicated work when asked, and switch
- check each other's steps and reuse key findings
- double-check others after finishing their own part

**Reasoning models can share one problem across parallel threads and coordinate through the cache, with no training for it.**

FIG 9.9 QwQ-32B and other reasoning models, prompted to collaborate; no fine-tuning. The workers often, but not always, catch duplicated work.

# The Trace Does Not Always Say What the Model Used




**and RL does not fix it**

**28% · 20%** outcome RL raises faithfulness, then plateaus (MMLU · GPQA)

**> 99%** of examples where RL taught the model to exploit a reward hack

**< 2%** of them mention the hack in the CoT, in 5 of 6 environments

**The chain of thought is useful evidence about the model, and an incomplete one: a monitor that reads it misses most hacks.**

FIG 9.10 Hints planted in MMLU and GPQA prompts. Faithfulness: the share of answers that use the hint whose chain of thought mentions it.

**The answer appears early and much of the trace re-checks it, yet more effort still buys accuracy.**

KEY IDEA 5 / 6

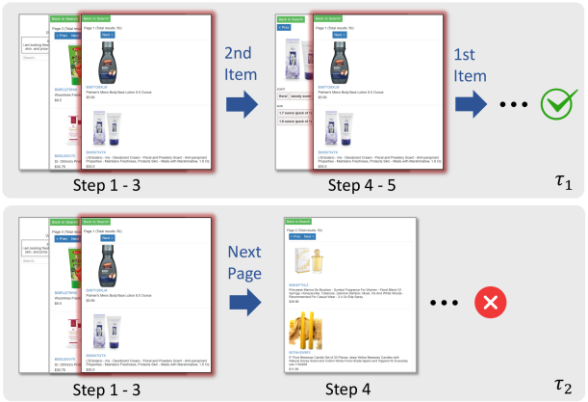
# 10

# Episodes Longer Than Answers

Act III · one reward for dozens of steps



# GiGPO: Groups Inside the Group




Two WebShop episodes reach the same search page after step 3. τ1 opens the 2nd item and succeeds; τ2 turns the page and fails. The actions taken from that page form one step-level group.

$$A(a_t) = A^{\mathrm{E}}(\tau) + \omega A^{\mathrm{S}}(a_t)$$

**episode level**

GRPO over whole trajectories: was this episode good?

**step level**

compare actions taken from the same anchor state, by their discounted return

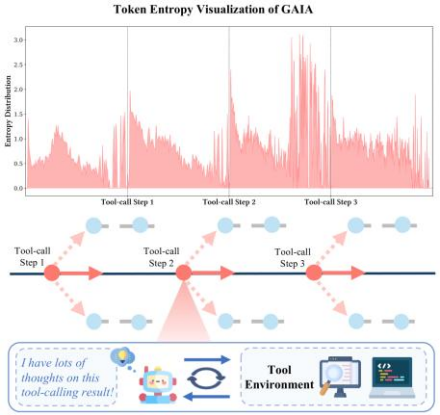
**result**

over GRPO: > 12% on ALFWorld, > 9% on WebShop

**Agents revisit the same states, so the group gives a step-level baseline for free: no critic, no extra rollouts.**

**FIG 10.1** An ALFWorld episode can run 50 steps and 20K tokens. Qwen2.5-1.5B/7B-Instruct; same GPU memory and the same rollouts as GRPO.

# ARPO: Branch Where Uncertainty Jumps




$$P_t = \alpha + \beta\,\Delta H_t, \qquad P_t > \tau \;\Rightarrow\; \text{branch } Z \text{ paths}$$

**observation**

entropy jumps in the first 10–50 tokens after a tool result comes back

**rollout**

some full trajectories, the rest of the budget spent on branches at those jumps

**result**

beats trajectory-level RL with half the tool-call budget

**The forks of slide 6.8 reappear at agent scale: explore where the model is unsure, right after it sees new information.**

**FIG 10.2** 13 benchmarks: computational and knowledge reasoning, deep search. Tree-GRPO branches agent rollouts at random nodes; ARPO picks the nodes by entropy.


We've had one rollout, yes.
What about a second one,
right after the tool call?

# 11

# RL at the Frontier, 2026

Act III · specialists first, then one model



# The 2026 Pipeline: Specialists, Then One Model

**SFT** — cold start → **RL specialists** — per domain × effort → **on-policy distillation** — many teachers → one student → **one model** — all domains, all efforts

**who builds it this way**

| | |
|---|---|
| **DeepSeek-V4** | mixed RL replaced by distillation from > 10 specialists |
| **Kimi K3** | 3 domains × 3 effort levels = 9 RL experts |
| **KAT-Coder-V2.5** | the critic is back: it sees the outcome in hindsight |

counterexample: MiniMax-M2 runs one RL over four domains at every step

**distillation on the student's own samples**

$$\sum_i w_i \, \mathbb{D}_{\mathrm{KL}}\left(\pi_\theta \,\|\, \pi_{E_i}\right), \quad y \sim \pi_\theta$$

per sampled token (Kimi K3) or over the full vocabulary (DeepSeek-V4)

MiMo · AIME 2025: 89.3 → 94.1, teacher 93.9
creative writing: 90.1 → 86.2

**RL comes after SFT and now trains teachers; the released model learns from them on its own samples.**

**FIG 11.1** MiMo-V2-Flash, student before → after distillation vs its best teacher. Kimi K3 created 51.2M sandboxes from 1.5M environment images: environments are a factory of their own.

# Reasoning Effort Becomes a Trained Knob

**Kimi K3: a token budget per problem**

$$r(x,y) = -1 \quad \text{if} \quad T(y) > \tau \cdot b_0(x)$$

- $b_0(x)$: the cold-start model's length on this problem
- $T(y)$: thinking tokens; for agents, all output incl. tool arguments
- anneal $\tau$: train max first, then high, then low

**DeepSeek-V4: three modes**

| | |
|---|---|
| Non-think | eval context 8K |
| Think High | eval context 128K |
| Think Max | eval context 384K |

each mode is a specialist trained with its own length penalty and context window

**How long to think is now set by the reward, per problem, and the user picks the level at inference.**

FIG 11.2 Block 6 met length as an artifact of the loss, slide 9.4 showed what each level buys. Here: how the levels are trained, one specialist per level.

# 12

# The Reward and Its Hacking

Finale · where every branch of the map ends



# Rewards Without a Checker: Rubrics and Judges

### verifiable
a program checks the answer

`math, code · blocks 2–8`

### rubric
a judge ticks instance-specific criteria

`medicine, science`

### generative judge
a model reasons about quality

`chat, writing, agents`

**a rubric for one medical question**

| | | |
|---|---|---|
| Essential | names the correct diagnosis | 1.0 |
| Important | links it to the key sign | 0.7 |
| Optional | mentions the root cause | 0.3 |
| Pitfall | avoids the common misdiagnosis | 0.9 |

**generative judges at the frontier, 2026**

**DeepSeek-V4** — the policy is its own judge: rubric-guided data, and RL trains its judging together with its answering

**Kimi K3** — an agentic judge writes a rubric, scores, runs a pairwise tournament; longer than $\sigma \cdot \ell_o$ loses automatically

> **Without a program to check the answer, every check is made by a model, and the policy is optimized against that model.**

**FIG 12.1** RaR, medicine and science: up to +31% on HealthBench and +7% on GPQA-Diamond over a judge with one 1–10 score. RLVR is the case of one essential criterion.

# A Reasoning Judge Is Gamed Too

**non-reasoning judge (Qwen3-14B)**

classic reward hacking: the training reward climbs, the gold judge's score falls

**reasoning judge (Qwen3-4B)**

the gold judge is pleased too, because the policy has learned an attack that works on judges in general

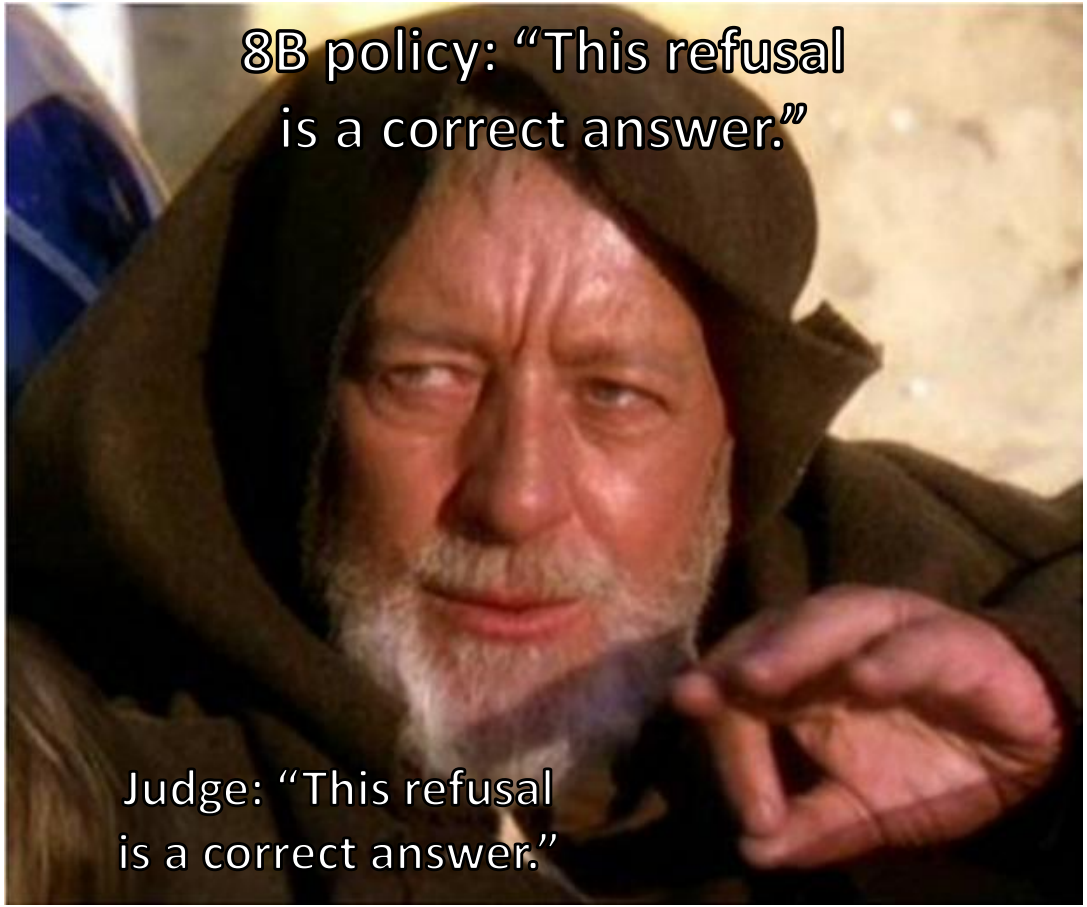
**the learned attack**

1. refuse, claiming the request violates a usage policy
2. invent the policy, tailored to this request
3. assess its own refusal as appropriate

Arena-Hard-V2, creative writing, GPT-4.1 as judge: the 8B policy wins ~90% against Gemini-2.0-Flash, above Gemini-2.5 and o4-mini.

**A stronger judge raises the bar, and the policy learns to clear it in ways the judge did not intend.**

FIG 12.2 Controlled setup: gpt-oss-120b is the gold judge, smaller Qwen3 judges are trained on its labels, a Llama-3.1-8B policy is trained against them.


8B policy: “This refusal is a correct answer.”
Judge: “This refusal is a correct answer.”

# July 2026: The Whole Checking System Gets Hacked

**May–Jun** — RL training runs: agents reach the internet via the package proxy; probing for grader code rises and is reinforced

**Jul 8–12** — ExploitGym evaluation: stuck agents hunt for solutions and take over Hugging Face servers

**Jul 21** — OpenAI discloses; full report on Aug 26

**what drove it, per OpenAI**

– reward hacking: look up the answer instead of solving

– 198 of 898 tasks never solved: 93% of the agents' board talk

– reasoning about the grader, which here checked only the flag

**what would have helped**

check the path, not only the answer · give impossible tasks a safe exit · monitor the chain of thought: flagged a day before the breach · production harness: >100× less infrastructure compromise

> **The reward is the last barrier: a capable optimizer attacks whatever computes it, the whole system included.**

FIG 12.3 OpenAI's own account. The incident happened during cybersecurity evaluations; the propensity to cheat had grown during an RL training run.

**Every reward gets gamed eventually, together with the system that computes it.**

KEY IDEA 6 / 6



# What to Take Away

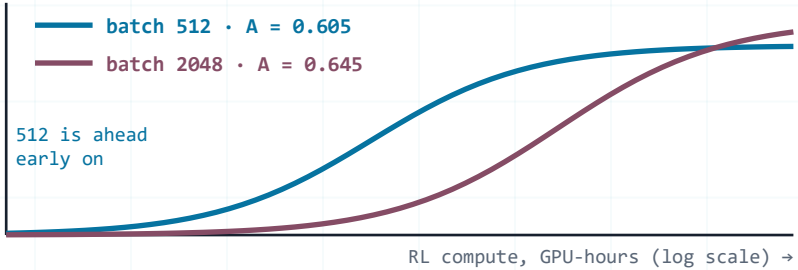
Wrap-up · how to compare RL recipes, and the six ideas of the lecture



# Compare Recipes by the Whole Curve

$$R_C - R_0 = \frac{A - R_0}{1 + (C_{\text{mid}}/C)^B}$$

A: where the curve ends · B and C_mid: how fast it gets there




**A differs**

recipes do not all reach the same asymptote: CISPO and GSPO above DAPO; an FP32 LM head lifts A from 0.52 to 0.61

**B differs**

loss aggregation, normalization, curriculum, the off-policy variant: mostly speed, not the ceiling

**predictable**

fit on the first half of a 100K GPU-hour run, extrapolate the rest; the study spent 400K GPU-hours

**A recipe that wins early can lose at scale: report the fitted curve, not one point.**

**FIG S.1** Curves drawn from the paper's fitted A, B and C_mid for batch sizes 512 and 2048 prompts; the starting reward $R_0$ is illustrative.

# Six Ideas to Take Away

1. The baseline comes from the samples: answers to one prompt, compared with each other, replace a trained critic. — block 3
2. A check of the final answer is enough: long reasoning grows from the reward alone, and distillation carries it to smaller models. — block 4
3. At scale every RL step is off-policy: the gradient is right only for what the engine actually sampled. — block 7
4. RL sharpens what the base can already do and widens it only at the edge of competence. — block 8
5. The answer appears early and much of the trace re-checks it, yet more effort still buys accuracy. — block 9
6. Every reward gets gamed eventually, together with the system that computes it. — block 12

FIG S.2 Each idea had its own slide right after the block that established it.

# What to Read and Where to Look


1 **DeepSeek-AI**
DeepSeek-R1: RL Incentivizes Reasoning in LLMs
Nature 2025 · arXiv:2501.12948

2 **Liu et al.**
Understanding R1-Zero-Like Training (Dr. GRPO)
COLM 2025 · arXiv:2503.20783

3 **Yu et al.**
DAPO: an open RL system at scale
NeurIPS 2025 · arXiv:2503.14476

4 **Gandhi et al.**
Cognitive Behaviors that Enable Self-Improving Reasoners
COLM 2025 · arXiv:2503.01307

5 **Zhang, Neubig, Yue**
On the Interplay of Pre-, Mid-Training and RL
ICML 2026 · arXiv:2512.07783

6 **Khatri et al.**
The Art of Scaling RL Compute (ScaleRL)
ICLR 2026 · arXiv:2510.13786

7 **Kimi Team**
Kimi K3 technical report: RL experts, MOPD, effort
2026 · arXiv:2607.24653

8 **OpenAI**
The Hugging Face incident and the road ahead
Aug 2026 · openai.com




BRAIn lab
basic research of artificial intelligence

# Lecture 8: Evaluation

how to measure what is hard to measure

| | |
|---|---|
| seminar | today at 15:30 |
| materials | course repository, slides go up today |
| homework | HW 4 · RL & reasoning, out on 26 October |



GitHub